Games and Simulations – Test 2 A

2024-2025

Date: 29/05/2025 Duration:1h15

Number: ___________ Name: ____

- Do not separate the sheets! You may use a pencil. Use the backs for drafts.
- You may bring a handwritten A4 page.
- The test has two groups: Group 1 – answer in the sheet; Group 2 – answer in Akindi.
- Each wrong answer in Group 2 has 10% of discount. If you don't answer it's worth 0.
- For all questions in this group, there is always an extra (implicit) option: "E - *no other option is correct*".

Group 1 – 8 points (answer in blank spaces)

1. Consider a scene where there is a planar surface (e.g. a table or a door) that reflects its environment as if it were a mirror, although still showing its material characteristics, such as base color, texture, specular reflection of light, etc.

   A. [**2.0**] Explain **in detail** how the stencil buffer can be used to correctly draw the scene. Your description should be a recipe (algorithm) with all the steps/phases clearly detailed.

   B. [**1.0**] Do you think that same surface can also be assigned a bump map? If affirmative, explain for what effect it could be used. If negative justify.

   C. [**1.0**] Suppose that instead of a planar surface we have a cube with the same characteristics described in A. How would you adapt the algorithm given as answer in A for this situation?

2. In certain game engines, such as Unity, shadow maps are used to generate real time shadows (for directional, point and spotlight light types). The algorithm works in two stages and uses the z-buffer to assist.

   A. [**2.0**] Explain how and why the technique works, as detailed as possible. In your explanation detail how the cast shadows (On/Off) and the Receive Shadows (On/Off) settings are used. In your opinion what are its main advantages over the use of shadow volumes? Can you also give an advantage of using shadow volumes instead of shadow maps?

   B. [**1.0**] Can you think of a way to dynamically determine a minimum shadow map resolution depending on the type of light (directional/spotlight/point) that tries to achieve a projected size for a pixel in the shadow map to be as big as pixel in the final image?

   C. [**1.0**] In Unity, the size limits (in pixels) for shadow maps differ for each type of light. Can you elaborate on the reasons for the relative sizes between the different types of lights?

      Size limits: directional - 4096x4096 / spotlights - 2048x2048 / point lights - 1024x1024.

Group 2 - 12 points (answer in bubble sheet)

1. A bump map defined in tangent(normal) space is a texture that holds:
   A. The components of the tangent space axes, defined in world space
   B. The coordinates of a vector in tangent space to be added to the interpolated normal vector
   C. The coordinates of the normal vector in tangent space
   D. The components of the world axes defined in tangent space

2. Assuming that access to each component of the bump map texture will return normalised values in [0,1] range for each channel, how would the looked up value (here identified as *v*) need to be changed before trying to convert it to camera/world space for lighting operations?
   A. (*v* - 0.5) * 2.0
   B. *v* * 2.0 + 1.0
   C. *v* * 255.0
   D. (*v* - 0.5) * 511.5

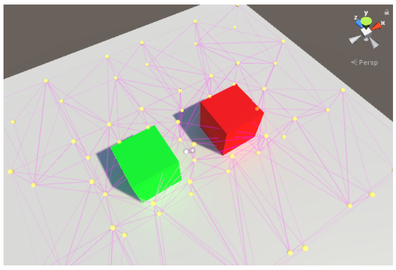
3. A depth map $\mathbf{D}$ (or displacement map) is a grayscale image that encodes the depth of a surface. White (1.0) values represent minimum depth (maximum height) and Black (0.0) values represent maximum depth (minimum height). Choose the expression that could be used to compute the value of the normal map **N** for at integer coordinates (i, j). Os índices (i, j) percorrem a textura ao longo dos seus eixos (u, v), respectivamente.
   A. $\mathbf{N}_{i,j} = (-\mathbf{D}_{i+1,j} + \mathbf{D}_{i,j}, \mathbf{D}_{i,j+1} - \mathbf{D}_{i,j}, 1.0)$
   B. $\mathbf{N}_{i,j} = (-\mathbf{D}_{i+1,j} + \mathbf{D}_{i,j}, -\mathbf{D}_{i,j+1} + \mathbf{D}_{i,j}, 1.0)$
   C. $\mathbf{N}_{i,j} = (\mathbf{D}_{i+1,j} - \mathbf{D}_{i,j}, \mathbf{D}_{i,j+1} - \mathbf{D}_{i,j}, 1.0)$
   D. $\mathbf{N}_{i,j} = (\mathbf{D}_{i+1,j} - \mathbf{D}_{i,j}, -\mathbf{D}_{i,j+1} + \mathbf{D}_{i,j}, 1.0)$

4. Transparent objects in a scene are usually drawn in a back to front order. Consider the algorithm studied that handles scenes with mixed (opaque and transparent) objects, when it is not possible to sort them in back to front order (e.g. objects crossing each other). What can be said about the **changes in the frame buffer** performed **on a single loop iteration**?
   A. Each transparent object can update different pixels on the frame buffer, but each pixel is modified by only one transparent object.
   B. All the pixels updated belong to the same transparent object.
   C. Each transparent object can update different pixels on the frame buffer, and different objects may change the same pixel.
   D. Each pixel covered by any transparent object gets updated.

5. After running the algorithm of the previous question (while keeping the contents of the z-buffer and the frame buffer), could extra opaque objects be rendered correctly?
   A. No, because extra opaque objects drawn under transparent objects already drawn would produce visually incorrect results.
   B. Yes, with no performance penalties in comparison to having them drawn as a step in the previous algorithm.
   C. Yes, but at the cost of a potential performance penalty (more write operations in the frame buffer than necessary)
   D. No, because extra opaque objects drawn over transparent objects already drawn would produce visually incorrect results.

6. **Light Probes** are available in Unity as a **baked** technique that stores information about the light travelling through a specific point in the scene in different directions (a **small set** of discretised directions

equally distributed in space to equally cover all possible directions). Several light probes are also distributed throughout the scene to capture the light transfers in different spots, as indicated in the figure by the dots. An object can thus sample information from nearby probes to account for light reaching its surface. Considering the Phong reflection model, for what types of reflection would that information be appropriate?



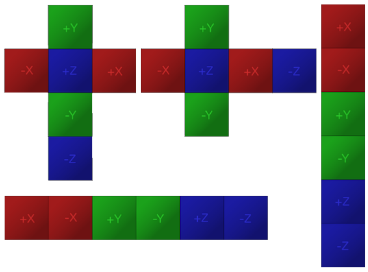
A. ambient reflection

B. specular reflection

C. diffuse and ambient reflection

D. all terms (ambient, diffuse and specular)

7. **Reflection Probes** can be used in Unity so that reflecting objects can show the reflected surroundings on its surface. With a cube map, 6 images can be captured from a specific point along the 6 principal directions that are then mapped to a different cube face, as illustrated in the figure, where several possible single texture layouts can are shown. Choose the correct sentence!



   A. When looking at an object that uses a reflection probe, it will not be possible to see the shadows casted on its surface by other objects.

   B. The object using the reflection probe will not cast shadows on other objects.

   C. When looking at nearby objects, we can see on their surfaces the reflection of the object using the reflection probe.

   D. The object using the reflection probe should be enclosed by the cube used to generate the cube map and each cube face can be used as the near clip plane in the generation of the corresponding environment map.

8. Regarding the shaders used for the object reflecting the environment on its surface, which sentence best describes what needs to be done?

   A. The shader will use the smallest component of the reflection vector R to choose the cube face to be sampled

   B. The shader will use the biggest component of the reflection vector R to choose the cube face to be sampled

   C. The shader will use the smallest component of the normal vector N to choose the cube face to be sampled

   D. The shader will use the biggest component of the normal vector N to choose the cube face to be sampled

9. **Complete the sentence in the most appropriate way!** Deferred shading can be used with substantial gains in scenes where…

   A. … lighting is costly and there are several surfaces projected on the same screen locations

   B. … lighting is costly and there is generally one surface projected on each pixel

   C. … cost of lighting is irrelevant but the scene has several surfaces projected on the same screen locations

   D. … cost of lighting is irrelevant and the scene has a large number of small polygons covering the view volume, but without being projected on the same screen locations

10. Regarding deferred shading in a scene with a large number of point lights with finite range, there is an algorithm that can be used to accelerate rendering with large speedups. What is the algorithm computing in the stencil buffer before the lighting stage of deferred rendering?

A. The number of lights in front of each pixel

B. The number of lights behind each pixel

C. The number of lights that reach each pixel

D. The distance to the closest light for the point surface projected at that pixel

11. Shader Model 4.0 exposed a new hardware feature of GPUs named Stream Output. This feature feature allows:

A. The pixels to be written into a texture in memory, after the pixel shader stage.

B. The stream of vertices to be output to a memory buffer, immediately before the rasteriser stage.

C. The incoming vertex stream to be captured into a memory buffer, immediately before the vertex shader stage.

D. The stream of pixels to be output to a memory buffer, after the rasteriser stage, and before the pixel shader stage.

12. The Hero in your game has a special gun with the special power of bringing peace to the world. This wonderful gun fires white flags, white doves and peace symbols into the air. The tip of the gun is located at point **P** in local coordinates of the gun object. The hierarchy is as represented in the figure. What is the expression that allows you to spawn the objects fired in world coordinates? (Note that each **Transform** component of a Game Object has updated transformations **localToWorldMatrix** and **worldToLocalMatrix**, that do to points what their name imply). Let _ltw and _wtl suffixes respectively represent those matrices, for a given game object (in prefix).

A. Gun_ltw * P

B. Gun_wtl * P

C. Hero_ltw * Torso_ltw * Arm_ltw * Hand_ltw * Gun_ltw * P

D. Hero_wtl * Torso_wtl * Arm_wtl * Hand_wtl * Gun_wtl * P